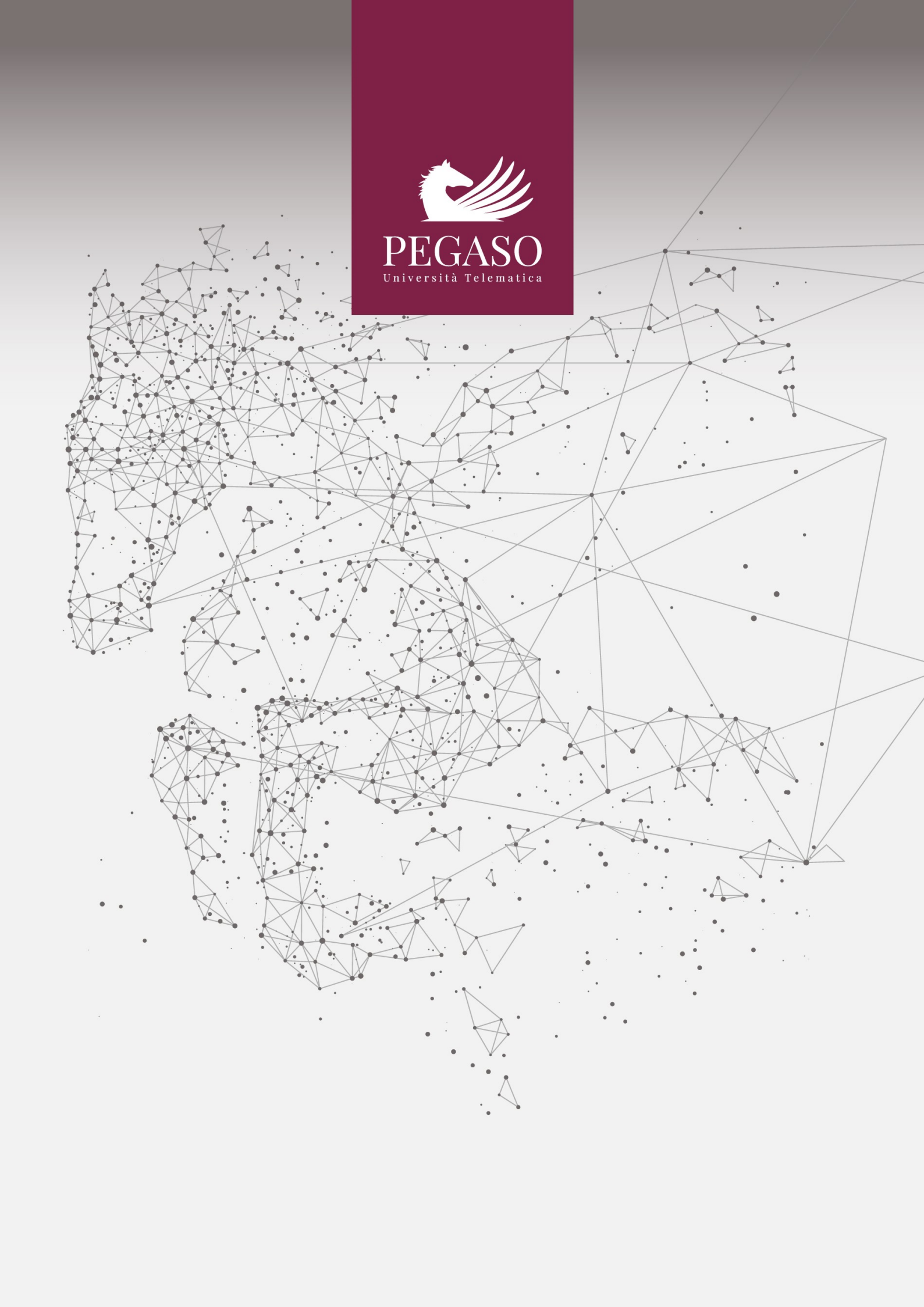

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE.....3
2. SCALABILITÀ 4
3. MIGRAZIONE IN SISTEMI ETEROGENEI 7
BIBLIOGRAFIA 9

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In determinate circostanze, il passaggio di programmi tra macchine diverse, a volte anche durante l'esecuzione, semplifica la progettazione di un sistema distribuito. In questa lezione verrà esaminata in modo più approfondito la migrazione del codice. Iniziamo esaminando i vari approcci alla migrazione del codice, seguita da una discussione su come gestire le risorse locali richieste da un programma di migrazione. La migrazione del codice in sistemi eterogenei è un problema particolarmente impegnativo, spesso oggetto di discussione.

Storicamente, la migrazione del codice nei sistemi distribuiti ha assunto la forma di migrazione dei processi, in cui tutto un processo è stato trasferito da un nodo all'altro. Spostare un processo in esecuzione su una macchina diversa è un'operazione costosa e complessa; quindi, ci deve essere una ragione convincente per farlo. La premessa fondamentale è che le prestazioni complessive del sistema possono essere migliorate spostando i processi da macchine con carichi pesanti a macchine con carichi leggeri. Il carico è spesso espresso in termini di lunghezza della coda della CPU o di utilizzo della CPU, ma vengono utilizzati anche indicatori di prestazioni aggiuntivi.

Tuttavia, invece di scaricare di lavoro le macchine, possiamo osservare che il codice può essere spostato per garantire che una macchina sia sufficientemente impegnata. In particolare, la migrazione di intere macchine virtuali con la loro suite di applicazioni a macchine con carico leggero al fine di ridurre il numero totale di nodi utilizzati è una pratica comune per ottimizzare il consumo energetico nei data center. È interessante notare che, sebbene la migrazione di macchine virtuali possa richiedere più risorse, l'attività stessa è significativamente meno complessa rispetto alla migrazione di un processo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Scalabilità

In generale, gli algoritmi di distribuzione del carico, che determinano l'allocazione e la ridistribuzione delle attività tra una raccolta di macchine, svolgono un ruolo cruciale nei sistemi ad alta intensità di calcolo. In molti sistemi distribuiti moderni, tuttavia, l'ottimizzazione della capacità di calcolo è meno problematica rispetto, ad esempio, alla riduzione al minimo della comunicazione. Inoltre, a causa dell'eterogeneità delle piattaforme e delle reti di computer sottostanti, il miglioramento delle prestazioni tramite la migrazione del codice è spesso basato su ragionamenti qualitativi piuttosto che su modelli matematici.

Si consideri un sistema client-server in cui il server gestisce un enorme database come esempio. Se un'applicazione client deve eseguire numerose operazioni di database su grandi quantità di dati, può essere preferibile spedire una parte dell'applicazione client al server e inviare solo i risultati in rete. In caso contrario, il trasferimento di dati dal server al client potrebbe sovraccaricare la rete. In questo caso, la migrazione del codice si basa sul concetto che è generalmente prudente elaborare i dati in prossimità della loro posizione.

È possibile migrare parti del server al client per lo stesso motivo. In numerose applicazioni di database interattive, ad esempio, i clienti devono compilare moduli che vengono poi tradotti in una serie di operazioni di database.

L'elaborazione del modulo sul lato client e l'invio al server solo del modulo compilato possono talvolta eliminare la necessità di un numero elevato di messaggi di rete di piccole dimensioni. Di conseguenza, il client percepisce prestazioni migliorate, mentre il server dedica meno tempo all'elaborazione dei moduli e alla comunicazione. Nel caso degli smartphone, l'esecuzione di codice sul dispositivo mobile piuttosto che sul server può essere l'unica opzione praticabile per ottenere prestazioni accettabili sia per il client che per il server.

Il supporto per la migrazione del codice può anche migliorare le prestazioni sfruttando il parallelismo, ma senza le complessità tipicamente associate alla programmazione parallela. Un esempio potrebbe essere la ricerca di informazioni su Internet. È relativamente semplice implementare una query di ricerca sotto forma di agente mobile, un piccolo programma che si sposta da un sito all'altro. Creando più copie del programma e inviandole a posizioni diverse, potremmo essere in grado di ottenere un incremento lineare di velocità rispetto all'utilizzo di una singola istanza del programma. Tuttavia, gli agenti mobili non hanno mai raggiunto un impiego diffuso perché mancavano di un chiaro vantaggio competitivo rispetto ad

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

altre tecnologie. È importante sottolineare che era praticamente impossibile consentire a questo tipo di codice mobile di operare in modo sicuro.

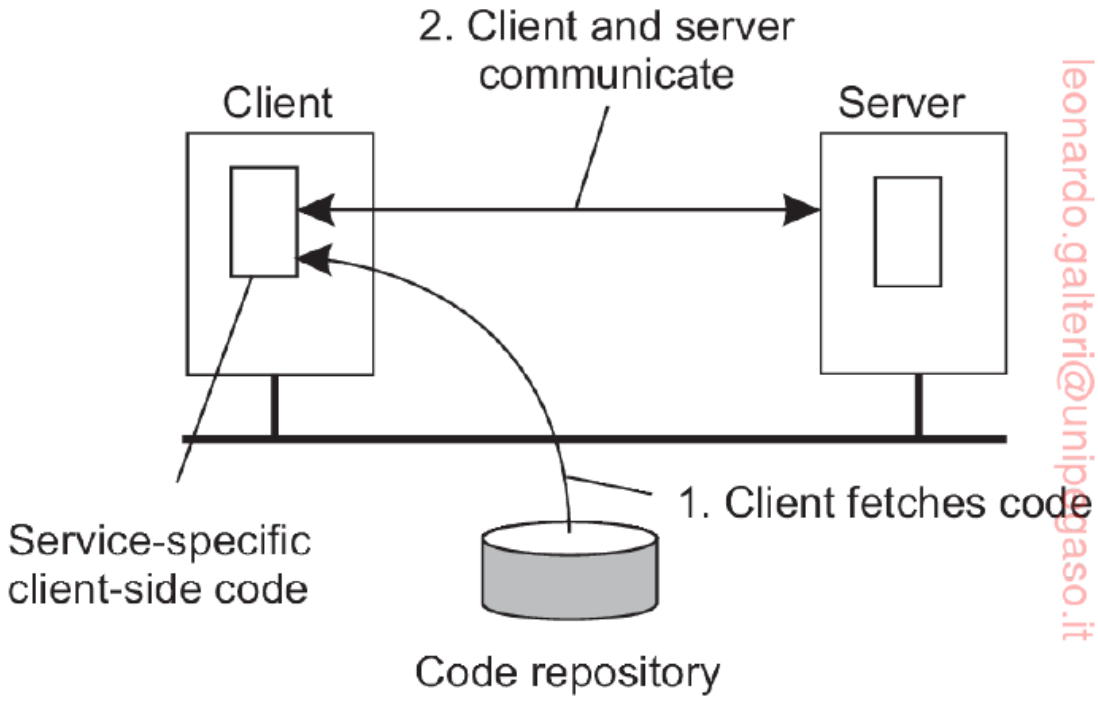
Oltre a migliorare le prestazioni, esistono altri motivi per supportare la migrazione del codice. La caratteristica più essenziale è la capacitò di essere flessibili. Il metodo convenzionale per lo sviluppo di applicazioni distribuite consiste nel dividere l'applicazione in sezioni distinte e determinare in anticipo dove verrà eseguita ciascuna sezione. Questa strategia, ad esempio, ha portato alla realizzazione di varie applicazioni client-server multilivello.

Tuttavia, se il codice può spostarsi tra macchine diverse, i sistemi distribuiti possono essere configurati dinamicamente. Si supponga, ad esempio, che un server implementi un'interfaccia di file system standard. Per l'accesso client remoto al file system, il server utilizza un protocollo proprietario. L'implementazione lato client dell'interfaccia del file system, basata su questo protocollo, dovrebbe in genere essere collegata all'applicazione client. Questa strategia richiede che il software sia facilmente accessibile al client durante lo sviluppo dell'applicazione client. Facoltativamente, il server potrebbe fornire l'implementazione del client solo quando strettamente richiesto, cioè quando il client si associa al server. Il client scarica quindi dinamicamente l'implementazione, esegue i passaggi di inizializzazione necessari e quindi chiama il server.

Questo principio è illustrato nella Figura 1. Questo modello di trasferimento dinamico del codice da una postazione remota richiede la standardizzazione del protocollo per il download e l'inizializzazione del codice. Inoltre, il codice scaricato deve essere eseguibile sul computer del client. In genere, gli script eseguiti in una macchina virtuale incorporata, ad esempio un Web browser, saranno sufficienti. Questo tipo di migrazione del codice è stato probabilmente cruciale per il successo del Web dinamico.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



leonardo.galteri@unipegaso.it

Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Il vantaggio del software in questione è che i client non hanno bisogno di avere tutto il software preinstallato per comunicare con i server. Invece, il software può essere spostato in base alle esigenze e rimosso quando non è più necessario. Finché le interfacce sono standardizzate, il protocollo client-server e la sua implementazione possono essere modificati con la frequenza desiderata. Le modifiche non influiranno sulle applicazioni client dipendenti dal server esistenti.

Ovviamente, ci sono anche degli svantaggi. La questione più seria riguarda la sicurezza. Non è sempre una buona idea presumere che il codice scaricato implementi solo l'interfaccia pubblicizzata durante l'accesso al disco rigido non protetto e non invii i dati più sensibili a chissà chi. Per fortuna, è ben chiaro come proteggere il client dal codice dannoso scaricato.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Migrazione in sistemi eterogenei

Fino a questo punto abbiamo implicitamente assunto che il codice migrato possa essere sempre facilmente eseguito sulla macchina di destinazione. Quando si lavora con sistemi omogenei, questa ipotesi è appropriata. Tuttavia, i sistemi distribuiti sono in genere costruiti su una raccolta di piattaforme eterogenee, ognuna con il proprio sistema operativo e la propria architettura di macchina.

In molti contesti, i problemi causati dall'eterogeneità sono identici a quelli causati dalla portabilità. Non sorprende che le soluzioni disponibili condividano molte somiglianze. Ad esempio, alla fine degli anni 1970, la generazione di codice intermedio indipendente dalla macchina per una macchina virtuale astratta era una soluzione semplice a molti dei problemi associati al porting di Pascal su macchine diverse.

Ovviamente, questa macchina virtuale doveva essere implementata su numerose piattaforme, ma avrebbe consentito ai programmi Pascal di funzionare ovunque. Sebbene questo semplice concetto sia stato ampiamente utilizzato per un certo numero di anni, non ha mai preso piede come soluzione generale ai problemi di portabilità in altre linguaggi, in particolare il C.

Circa venticinque anni dopo, i linguaggi di scripting e i linguaggi altamente portabili come Java vengono utilizzati per affrontare la migrazione del codice in sistemi eterogenei.

In sostanza, queste soluzioni impiegano la stessa strategia del porting di Pascal. Tutte queste soluzioni si basano su una macchina virtuale (di processo) che interpreta direttamente il codice sorgente (come nel caso dei linguaggi di scripting) o interpreta il codice intermedio generato da un compilatore (come in Java).

Ulteriori progressi hanno ridotto la dipendenza dai linguaggi di programmazione. In particolare, sono state proposte soluzioni per migrare non solo i processi ma anche interi ambienti informatici. Il concetto fondamentale è quello di partizionare l'ambiente generale e fornire i processi nella stessa sezione con la propria prospettiva sul proprio ambiente informatico. Questo partizionamento è ottenuto per mezzo di monitor di macchine virtuali che eseguono un sistema operativo e una raccolta di applicazioni.

Con la migrazione di macchine virtuali, è possibile disaccoppiare un ambiente di elaborazione dal sistema sottostante e migrarlo su un'altra macchina. Uno dei principali vantaggi di questo approccio è che i processi possono rimanere all'oscuro della migrazione stessa: la loro esecuzione non deve essere interrotta e non dovrebbero in alcun modo incontrare problemi con le risorse utilizzate. Questi ultimi o migrano con un processo, o il modo in cui un processo accede a una risorsa non è influenzato.

Ad esempio, alcuni ricercatori si sono concentrati sulla migrazione in tempo reale di un sistema operativo virtualizzato, che è tipicamente vantaggioso in un cluster di server strettamente accoppiati

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

tramite una singola rete locale condivisa. In queste condizioni, la migrazione presenta due sfide significative: la migrazione dell'intera immagine di memoria e la migrazione delle associazioni alle risorse locali.

Per quanto riguarda la prima questione, ci sono essenzialmente tre approcci (combinabili) alla migrazione:

- Invio delle pagine di memoria alla nuova macchina e invio di quelle successivamente modificate durante la procedura di migrazione.
- Arresto della macchina virtuale corrente, trasferimento della memoria e avvio della nuova macchina virtuale.
- Consentire alla nuova macchina virtuale di inserire nuove pagine in base alle esigenze, ovvero consentire l'avvio immediato dei processi nella nuova macchina virtuale e copiare le pagine di memoria su richiesta.

La seconda opzione potrebbe causare tempi di inattività inaccettabili se la macchina virtuale di cui viene eseguita la migrazione esegue un servizio attivo o uno che fornisce un servizio continuo. D'altra parte, un approccio on-demand puro, come rappresentato dalla terza opzione, può non solo allungare significativamente il periodo di migrazione, ma anche comportare prestazioni scadenti perché richiede molto tempo per trasferire il working set dei processi migrati alla nuova macchina.

Esiste come alternativa un approccio pre-copia che combina la prima opzione con una breve fase di stop-and-copy, rappresentata dalla seconda opzione. Si può rilevare che questa combinazione può comportare interruzioni del servizio estremamente basse.

La gestione delle risorse locali è semplificata quando è coinvolto solo un server cluster. Innanzitutto, poiché esiste una sola rete, è necessario solo annunciare la nuova associazione di indirizzi da rete a MAC in modo che i client possano contattare i processi migrati nell'interfaccia di rete corretta.

Infine, se si presuppone che l'archiviazione sia fornita su un livello separato la migrazione dell'associazione ai file è altrettanto semplice, in quanto consiste principalmente nel ristabilire le connessioni di rete.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Milojicic D., Douglis F., Paindaveine Y., Wheeler R., and Zhou S. Process Migration, 2000.
- Kumar K., Liu J., Lu Y.-H., and Bhargava B. A Survey of Computation Offloading for Mobile Systems. Mobile Networks and Applications, ACM Computing Surveys, 2013.
- Carzaniga A., Picco G. P., and Vigna G. Is Code Still Moving Around? Looking Back at a Decade of Code Mobility, 2007.
- Barron D. Pascal – The Language and its Implementation, 1984.
- Medina V. and Garcia J. A Survey of Migration Mechanisms of Virtual Machines. ACM Computing Surveys, 2014.
- Live Migration of Virtual Machines. In 2nd Symposium on Networked Systems Design and Implementation Live Migration of Virtual Machines, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).